COP290-Assignment 3**: LevelDB**

https://github.com/kirthikiron/COP290-LevelDB.git

Report

prepared by:

Peduru Kirthi Kiron 2020CS10366 Pragya Malaiya 2020CS10367

A. Analysis of LevelDB Write Path and Compaction Mechanism

1. The Write Path

Entry Point and Batching: The write path begins when a user application calls *DB::Put* or *DB::Delete* (located in *db/db_impl.cc*). Instead of immediately touching the database, these functions simply create a *WriteBatch* object, append the single operation to it (with a delete just being a write that inserts a tombstone marker), and pass it down to the core engine function: *DBImpl::Write*.

Queueing and The Lock: Once inside *DBImpl::Write*, the thread does not just rush to memory. It is assigned a *Writer* and pushed to the back of a double-ended *writers_* queue. The thread grabs the global mutex lock, but if it is not at the front of the queue, it enters a while loop and calls *Wait()*. This puts the thread to sleep and drops the lock so other incoming writes can also queue up.

Admission Control: When a thread finally reaches the head of the queue, it becomes the "leader". Before executing any writes, it checks the health of the engine by calling *MakeRoomForWrite*. This acts as a throttle. If Level 0 has 8 or more files, it sleeps to let background compaction catch up. If Level 0 has 12 files, it blocks completely. If the active *mem_* table is full, it rotates it into the immutable *imm_*, creates a new Write-Ahead Log (.log file), and triggers *MaybeScheduleCompaction()* to start flushing data to disk.

Group Commit and RAM Insertion: With room guaranteed, the leader thread executes a group commit. Instead of just writing its own batch, it calls *BuildBatchGroup* to scoop up the batches from all the other sleeping threads behind it in the queue. It drops the global lock and writes this massive combined payload to the hard drive's Write-Ahead Log via *log_->AddRecord()*. Writing one large block to disk is significantly faster than writing many tiny ones. Finally, it uses *WriteBatchInternal::InsertInto* to unpack the batch and insert the keys into the active *MemTable* (a SkipList in RAM). The leader then reacquires the lock, marks the sleeping threads' tickets as done, and wakes them up.

2. The Read Path

The Sequence Trigger and State Freezing: A read operation starts at *DBImpl::Get*. Because LevelDB uses versioning, the very first thing it needs is a *SequenceNumber* (either from a user snapshot, or just the global clock's current time) to pretend it is looking at the database at that exact instant. The thread then briefly grabs the global lock to "freeze" the system state. It grabs the pointers for *mem_*, *imm_*, and the current *Version* (the map of disk files). It calls *Ref()* on these three objects to increment their reference count, which protects them from being deleted by background compactions while the read is happening. It then unfreezes the system by dropping the lock.

The Lookup Flow: The target key and sequence number are combined into a *LookupKey*. The search strictly follows the flow of data from newest to oldest. It first checks the active memory via *mem_->Get()*. If missing, it checks the frozen memory via *imm_->Get()*.

Disk Traversal: If the key is not in RAM, the search falls back to the hard drive via *Version::Get*. It first checks Level 0. Because L0 files are hastily flushed from RAM, their key ranges overlap, so LevelDB must search all overlapping L0 files from newest to oldest. If it drops to Level 1 and below, the files are strictly sorted and non-overlapping, so the engine can use a very fast binary search (*FindFile*) to pick the correct file. It then uses *TableCache::Get* to pull the data block.

Iterators: When a user wants a sequential view of the database via *NewIterator()*, the engine calls *NewInternalIterator()*. This gathers child iterators from *mem_*, *imm_*, and the *Version* disk files, and feeds them all into a *MergingIterator* (which acts like a multi-way merge sort). This raw stream is then wrapped in a *DBIter*, which acts as a filter to hide internal mechanics from the user—skipping older key versions, hiding deletion tombstones, and enforcing snapshot sequence numbers.

3. The Compaction Mechanism

The Compaction Trigger: Without compaction, the disk would fill entirely with deleted records and old versions, causing read latency to skyrocket. To prevent this, the engine continuously runs *MaybeScheduleCompaction()* which spawns a background thread if it detects work is needed.

Minor Compaction (RAM to Disk): The simplest compaction happens if *imm_* is present. The background thread executes *CompactMemTable()*. It translates the immutable

memtable into an iterator, passes it to *BuildTable*, and literally writes a brand new, sorted Level 0 *SSTable* to the hard drive. Once written, *VersionSet::LogAndApply* writes a record to the MANIFEST file to make the new file officially visible to the database. It then cleans up by setting the *imm_* pointer to null and calling *Unref()*.

Major Compaction (Disk Level Merging): Every disk level has a compaction score. Level 0 is judged by file count (triggers at 4+ files), while Level 1 and below trigger based on exponentially increasing physical size limits. If a level exceeds its limit, *VersionSet::PickCompaction* grabs a file from Level L, and finds all overlapping files in Level L+1.

The Merge and Purging Drops: The core work happens in *DBImpl::DoCompactionWork*. A *MergingIterator* iterates over the chosen files in alphabetical order. This is where the vital data drops occur. First is the "Older Version Drop": if it sees a newer copy of a key, it drops the old ones (unless an active snapshot needs them). Second is the "Tombstone Drop": if it sees a deletion marker, it checks deeper levels using *IsBaseLevelForKey*. If no older copies exist in deeper levels, the tombstone itself is removed. The surviving keys are written to new Level L+1 files, which are split at roughly 2MB to prevent them from spanning too huge of a key range. Finally, *InstallCompactionResults* links the new files to the manifest, commits via *LogAndApply*, and deletes the old input files.

B. Design and implementation of the RangeScan API

Interface Modifications

LevelDB codebase placed the class declarations separately from the definitions so we began by modifying the abstract base class *DB* in *include/leveldb/db.h*. We added *Scan* as a pure virtual function alongside *Get*, *Put*, and *Delete*. We then added the corresponding override declaration to the concrete *DBImpl* class inside *db/db_impl.h*.

Implementation Logic and the *DBIter* Abstraction

The actual logic was implemented in *db/db_impl.cc*. Initially, scanning across the database seemed complex: data is scattered between the active *MemTable*, the immutable *imm_*, and overlapping disk files in Level 0, all while containing deleted keys and obsolete versions.

However, our job was made significantly simpler by LevelDB's existing iterator design. By calling *this->NewIterator(options)*, the engine internally constructs a *MergingIterator*

wrapped in a *DBIter*. As analyzed in Phase 1, *DBIter* automatically handles all the heavy lifting—it hides deletion tombstones, enforces snapshot sequence visibility, and drops older duplicate versions of keys. Because of this powerful abstraction, our *Scan* implementation didn't need to manually interact with any memory structures or SSTables. We simply acted as a driver for the clean iterator interface.

For resource safety, we wrapped the raw iterator pointer in a *std::unique_ptr<Iterator>*. This modern C++ practice ensures the iterator is automatically destroyed when the function returns, preventing memory leaks without needing manual *delete* calls. The core logic consists of seeking to the lower bound (*it->Seek(start_key)*) and running a *while* loop. To strictly enforce the half-open interval [*start_key*, *end_key*) required by the assignment, the loop condition checks *it->Valid()* and uses the lexicographical *compare()* function to ensure the current key is strictly less than the *end_key* (*it->key().compare(end_key) < 0*). If the condition is met, the key and value strings are pushed into the result vector, and *it->Next()* is called.

Resolving the Compilation Error caused by *ModelDB* class

During the build phase, we encountered a compilation error tied to LevelDB's internal test suite. By adding a pure virtual function to the *DB* base class, we inadvertently broke *db/db_test.cc*. This file contains a dummy mock class named *ModelDB* that inherits from *DB* for unit testing. Because *ModelDB* lacked an implementation for our new *Scan* function, the compiler treated it as an abstract type and refused to build the test suite. To resolve this, we modified *db/db_test.cc* by adding a dummy override for *Scan* inside *ModelDB* that simply returns *Status::NotSupported()*. This satisfied the C++ inheritance contract and allowed the parallel build process to complete successfully.

Verification

To verify the implementation, we wrote a standalone C++ script (*test_scan.cpp*) that linked against the compiled LevelDB engine. We populated a fresh database directory with a lexicographically sorted set of keys and executed a range scan. The output successfully retrieved the correct subset of keys, proving that the iterator properly traversed the data structures and respected the exclusive upper bound.

C. Design and Implementation of Range Delete API

Setting Up the Interface

Just like in Range Scan, the first step was wiring up the public API. We added the *DeleteRange(start_key, end_key)* function to the abstract *DB* base class in *include/leveldb/db.h* and declared the override in the *DBImpl* class inside *db/db_impl.h*. Also of course added a dummy function definition in *db/db_test.cc*'s *ModelDB* class. Since we had just done this for the scan API, getting the declarations in place was pretty straightforward.

Not Needing to Alter Compaction Mechanism

When the assignment PDF specifically noted that keys in the deleted range must be discarded when *SSTables* are merged, at first glance, this made it sound like we needed to dive into *db/compaction.cc* and edit it for this *DeleteRange* implementation. However, after reviewing how standard deletes work in Phase A, we realized we didn't need to change the compaction implementation at all. LevelDB's background compaction thread (*DBImpl::DoCompactionWork*) is already built to permanently erase any key that has a tombstone attached to it. Our design decision was to just drop tombstones on the keys in the range, like the original *Delete* operation and let the existing major and minor compaction mechanisms do the actual garbage collection later.

Using WriteBatch for Speed and Atomicity

For the core logic in *db/db_impl.cc*, we used a *std::unique_ptr<Iterator>* to traverse the [*start_key*, *end_key*) interval. Originally, we considered just calling the standard *Delete()* function for each key inside the while loop. While that would technically work, doing it one by one would mean acquiring the global mutex lock and forcing a disk write to the log file for every single key—which would absolutely kill the engine's performance.

Instead, we decided to use a *WriteBatch*. Inside the loop, we just call *batch.Delete(it->key())* to pack all the tombstones together. Once the loop finishes, we send the entire batch to the database in a single *Write()* call. Beyond just making it drastically faster, this guarantees atomicity. If the system suffers a sudden power loss halfway through a massive range deletion, the database won't be stuck in a corrupted, partially-deleted state. It's all-or-nothing.

Looping Logic

The traversal loop works exactly like the scan function: we jump to the start using *it->Seek(start_key)* and keep iterating as long as *it->Valid()* is true and the *compare()* function confirms the current key is strictly less than the *end_key*.

Acknowledgement. There was one suggestion given to us while debugging with Gemini. That is to add an *if (!it->status().ok())* check right after the loop finishes, but before actually committing the batch. Because a bad disk sector or corrupted file could cause the iterator to crash and abort the loop early, which apparently is common. Without this conditional check, we would blindly commit the *WriteBatch*, resulting in a partial deletion, while the program incorrectly assumed it succeeded. This check safely aborts the atomic write and reports the I/O error if the disk read failed.

D. Design and Implementation of Manual Full Compaction Mechanism

Background and Learnings for (and from) this Part

This final phase was more complex than the previous two implementations. LevelDB relies on an internal scheduling policy to perform compactions asynchronously in a background thread. We use this background process, force it to process all levels sequentially, block foreground operations until completion, and accurately track custom metrics (compaction counts and file counts) that LevelDB does not natively record.

This phase provided significant insights into LevelDB's optimization strategies and internal state management:

- **Leveraging Internal Testing APIs for Synchronization:** While investigating how to manually trigger the background thread, we discovered two public methods: *TEST_CompactMemTable()* and *TEST_CompactRange()*. They are testing APIs designed by developers to force compaction states. *TEST_CompactRange(level, begin, end)* enabled us to force a major compaction at a highly specific level. Crucially, these functions encapsulate all the complex condition-variable logic required to block the foreground thread until the background worker finishes. Using them completely abstracted away the highly error-prone task of manually managing the database's global mutex, making the implementation of a strictly synchronous loop remarkably clean.
- The "Minor Compaction" Path: We learned that memory flushes (MemTable to Level 0) use a fundamentally different code path (*WriteLevel0Table*) than standard disk

sweeps (*DoCompactionWork*). Tracking full system activity requires intercepting both.

One of our design decision was to include the MemTable Compaction as a part of the Manual Full Compaction, and hence made it so the Compaction stats that are printed is evaluated by considering stats due to Minor Compactions too.

Upgrading the Statistics

To track the metrics required by the assignment, we first extended the *CompactionStats* struct in *db/db_impl.h* by adding *compactions_executed*, *input_files*, and *output_files*. Instead of trying to count files in our foreground loop—which would risk thread-safety issues—we injected logic directly into the background worker. Inside *db/db_impl.cc*, we modified *DBImpl::DoCompactionWork* to calculate the input arrays and output vectors, adding them to the compaction receipt right before it gets logged to the database.

The *stats_* array of *DBImpl* class

LevelDB maintains a *stats_* array that acts as a lifetime odometer, permanently keeping a record of overall work due to all background compactions compactions at a level (destination level) since the database was opened. If we simply printed this array after our manual compaction, it would include weeks of background noise. To isolate our function's impact, we implemented a snapshot mechanism. At the start of *ForceFullCompaction()*, we aggregate all 7 level buckets into a *start_stats* baseline. After the work finishes, we take an *end_stats* snapshot and compute the delta, guaranteeing mathematical soundness.

RAM Flushes and the Loop Implementation

A true "full" compaction must include recent writes currently sitting in RAM. Therefore, the function execution strictly follows this order:

- Minor Compaction: We call *TEST_CompactMemTable()* to flush the active MemTable to the disk. To ensure this was tracked in our stats, we injected our custom file counters into the *WriteLevel0Table* function.
- Synchronous Major Compaction: We execute a for loop from Level 0 down to Level 5. Inside the loop, we invoke *TEST_CompactRange(level, nullptr, nullptr)*, with *nullptr* instructing the engine to sweep the entire alphabetical dictionary.

Because both *TEST_CompactMemTable* and *TEST_CompactRange* utilize internal condition variables, they inherently block our foreground thread (and any concurrent application read/write requests) until the background worker confirms the hard drive I/O is 100%

complete. Once the loop finishes, the function calculates the delta, prints the terminal statistics table, and safely returns *Status::OK()*.

E. Evaluation of Implemented Features

1. Evaluation of Range Operations (Scan and Delete) To evaluate the *RangeScan* and *DeleteRange* APIs, we developed custom C++ test scripts to verify strict adherence to the half-open interval [start_key, end_key) requirement.

- **Boundary Testing:** We verified that keys exactly matching the start_key were included in the operations, while keys exactly matching the end_key were successfully excluded. Edge cases, such as querying empty ranges or bounds outside the database's existing keys, were tested to ensure the iterator gracefully halted without causing segmentation faults.
- **Atomicity Verification:** For *DeleteRange*, we evaluated our WriteBatch design by ensuring that massive range deletions executed as a single atomic unit. By utilizing batching, we confirmed that the engine avoids the severe performance degradation associated with acquiring the global mutex and writing to the log file for every individual key deletion.

2. Evaluation of Manual Full Compaction To evaluate the *ForceFullCompaction* API and our custom statistics tracker, we subjected the engine to two distinct testing environments: a high-load test and a micro-test to analyze edge-case routing.

Large Input Test: We wrote a test script that inserted 100,000 records with intentionally large string values to completely fill the *MemTable* and force natural background data cascades. Following the insertion, we invoked our manual API. The engine successfully blocked the foreground thread, performed the synchronous sweep, and yielded the following isolated delta statistics:

- **Compactions Executed:** 18
- **Input Files:** 19
- **Output Files:** 21
- **Total Bytes Read:** 34,664,679
- **Total Bytes Written:** 35,754,434

Analysis: These results perfectly align with expected engine behavior. The ~35 MB of disk I/O accurately reflects the size of our 100,000-record payload plus LevelDB's internal metadata (Bloom filters and index blocks). An execution count of 18 over 19 input files confirms that the *TEST_CompactRange* loop successfully crawled through the heavily populated LSM-tree, isolating overlapping files and physically rewriting them to deeper levels.

Micro-Test : To rigorously evaluate the mathematical accuracy of our delta snapshot logic without background noise, we tested the API against an empty database using only 3 small records.

During this evaluation, we discovered a critical architectural constraint: LevelDB typically uses a "Trivial Move" optimization to push files to deeper levels without reading or writing physical bytes if there is no overlapping data. However, our evaluation proved that LevelDB explicitly disables this optimization during manual compactions (via an !is_manual conditional guard).

Consequently, after the initial RAM flush dropped our single tiny file into Level 2, our manual loop physically forced the engine to execute *DoCompactionWork* for levels 2→3, 3→4, 4→5, and 5→6. Our statistics tracker flawlessly captured this edge case, reporting exactly 5 compactions executed (1 minor + 4 major) and exactly 4 input files, mathematically proving that our injection into the background worker accurately traces physical execution paths level-by-level.